

SnifferDog: Comprehensively Learning Heterogeneous Features of Network Traffic to Identify Malicious Flows

Xi Luo[✉], Lihua Yin[✉], Hongyu Yang[✉], Zeyan Liu, Weizhe Chen[✉], Shijie Jia[✉], Bo Luo[✉], *Member, IEEE*, and Hongli Xiang[✉]

Abstract—Deep learning has recently attracted significant attention in the field of network intrusion detection. Despite a substantial number of efforts have been made, previous works struggle to comprehensively learn the features of network traffic, resulting in inconsistent performance across various environments and attacks. To address these limitation, this study presents SnifferDog, a novel network attack detection system that takes raw packets as input and rationally extracts and integrates heterogeneous features involved in packets, flows and topology. It formats the packets and flows concurrently to achieve a high-level throughput for feature learning. Then, a flow pretraining model consisting of a LSTM, a self-attention and cross-attention layers is developed to learn both sequential and nonsequential inter packet relation features as initial flow vectors. Subsequently, a node-to-node and a node-to-edge attention layers are implemented to enhance an inductive GNN model that dynamically embeds the flow-to-flow and flow-to-topology relation features into the flow vectors. The resulting flow vectors involve comprehensive information of packet-to-packet, flow-to-flow and flow-to-topology relations, enabling high detection performance. In-lab experiments across eight datasets from diverse environments demonstrate SnifferDog’s superior effectiveness over existing solutions. A scalable prototype deployed in our institute’s network achieves a false positive rate of only 0.08%, validating SnifferDog’s practicality in real-world scenarios.

Index Terms—NIDS, GNN, heterogeneous feature fusion, attention mechanism.

I. INTRODUCTION

NOWADAYS, cyberattacks have infiltrated various critical applications, including Internet of Things (IoT) systems, well-protected corporate networks, and industrial infrastructures, causing significant damage to users worldwide [1]. Network-based intrusion detection systems (NIDSs) have emerged as effective security solutions [2]. However, traditional NIDSs still heavily rely on signatures and handcrafted

rules. The processes of capturing and analyzing attack samples are time-consuming, and the detection rates remain low, as each rule is effective only against specific attack types.

Behavior-based NIDSs have increasingly gained preference among security researchers due to their ability to support more generalized attack detection with higher detection rates compared to signature and handcrafted rule-based systems. Relevant technologies can be categorized into two types: traditional machine learning (TML) and deep learning (DL). DL-based approaches are foundational for current network intrusion detection tasks, while TML methods are limited by a strong reliance on manual feature engineering and exhibits reduced effectiveness in extracting and learning network features [3], [4], [5]. Many studies, including [5], [6], [7], [8], [9], [10], [11], and [12], have demonstrated the efficacy of DL-based technologies in areas such as DDoS attack detection, botnet traffic identification, and IoT attack traffic analysis.

Nevertheless, existing DL-based methods focus only on representing partial network traffic features. Specifically, they consider only a part of packet-level, flow-level, and topology-level characteristics instead of fusing them in whole. For instance, many studies [13], [14], [15], [16], [17], [18], [19], [20] utilized statistical features derived from packet headers, neglecting the payload and thus losing information. Some researches like [21], [22], and [23] attempted to represent entire packets at the byte level, but they were computationally intensive due to the bi-gram byte embeddings (65,535 dimension per byte), byte graph learning and large language models they used. Moreover, they failed to learn the features capturing the relations between packets, flows and topology. Packets are related when application-layer data is split across multiple packets for network transfer, while flows and topology become interrelated during multi-step or distributed attacks. These relations are not easily extracted as statistical values based on expertise. The SOTA works [12], [24] took some relations into account, fully integrate them. Additionally, they used only a limited set of statistical features, such as timestamps and packet sizes, as the raw input information to ensure system throughput, resulting in unstable detection efficiency in real-world implementation due to substantial variations in feature distributions observed across varying network environments and cyber-attacks.

Unfortunately, efficiently and comprehensively extracting these features without expertise remains highly challenging, primarily due to two major obstacles. **1) Data Scale:** Using raw packet bits (approximately 1500×8 dimensions) instead of

Received 29 October 2024; revised 4 June 2025; accepted 2 October 2025. Date of publication 13 October 2025; date of current version 4 November 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3104100. The associate editor coordinating the review of this article and approving it for publication was Dr. Abdallah Shami. (Corresponding author: Lihua Yin.)

Xi Luo, Lihua Yin, Hongyu Yang, Weizhe Chen, and Hongli Xiang are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China (e-mail: xluo@gzhu.edu.cn; yinlh@gzhu.edu.cn; binball@gzhu.edu.cn; chenwz@gzhu.edu.cn; xamxxegen@gzhu.edu.cn).

Zeyan Liu and Bo Luo are with the Department of EECS and the Institute of Information Sciences, The University of Kansas, Lawrence, KS 66045 USA (e-mail: zyliu@ku.edu; bluo@ku.edu).

Shijie Jia is with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100195, China (e-mail: jiashijie@iie.ac.cn).

Digital Object Identifier 10.1109/TIFS.2025.3620640

statistical features (around 80 dimensions) results in a 150-fold increase in input dimensionality, creating prohibitive computational burdens. **2) Feature Heterogeneity:** Packets, flows, and topology exhibit distinct data structures. While statistical features of them can be directly extracted through basic mathematical operations, relational features among them necessitate specialized models for training and learning. These involve the representation and integration of heterogeneous features from raw bits, sequential data, and graph-based formats. Particularly for graph-based formats, multiple flows may exist between two communication devices, forming a graph structure where multiple edges connect two nodes. This structure, referred to as a flow graph in this study, is more complex than the typical single-edge-connected graph structure, making the learning and fusing of features more challenging.

In this work, we present a novel network intrusion detection system named SnifferDog,¹ which identifies malicious flows by comprehensively and efficiently fusing the features involved in packets, flows, topology, and their relations, without manual intervention. For clarity, we categorize the relations into three types: packet-to-packet, flow-to-flow and flow-to-topology. The packet-to-packet relation denotes the contextual relevance among the packets in a single flow inherited from the application layer data. The flow-to-flow relation represents the collaborative relevance when flows are involved in a benign application or an attack behavior. The flow-to-topology relation refers to the relevance between flows and communication devices in a network environment, as attackers usually assign dedicated malicious tasks to specific devices.

SnifferDog is composed of two critical components: **1) Flow pretraining** normalizes packet and flow data formats in a concurrent mode to support batch processing, thus enhancing the throughput; and it constructs a self-supervised model to generate initial flow embeddings by learning feature of the packets and packet-to-packet relation. **2) Flow graph learning** takes the initial flow embeddings as initial edge embeddings, and implements a novel Graph Neural Network (GNN) model for the flow graph, which can fuse the heterogeneous features of flow-to-flow and flow-to-topology relations concurrently into flow embeddings.

Flow pretraining. This component captures packets from the gateway device and generates initial flow vectors, thereby introducing two main performance challenges. 1) the processing speed during packet capturing and normalization and 2) the computational efficiency of the self-supervised learning model. For the first issue, SnifferDog increases the buffer size of the packet capture tool (libpcap [25]), thereby boosting the number of concurrent threads during packet normalization processing. For the second issue, SnifferDog analyzes the smoothing points in the entropy distribution of packets to minimize the number of bits used in each packet while preserving sufficient information for attack detection. Then, SnifferDog transforms each flow into a sequence of slices consisting of a fixed number of packet vectors, through a sliding window mechanism. The self-supervised learning model is an encoder-decoder model designed based on the Long Short Term Memory (LSTM) network and attention mechanism to learn both the sequential (for plaintext traffic) and nonsequential (for

encrypted traffic) packet relations. We chose LSTM rather than the popular Transformer architecture here because 1) a large portion of flows are short and contain only a few packets [24], while the Transformer is designed for long sequence, and the flows are split into a sequence of short slices in SnifferDog to accelerate the learning process; 2) the multi-head attention mechanism in the Transformer increases the computational demand, which deteriorates the throughput.

Flow graph learning. We formulate the flow graph as an edge-centric structure, in which each edge has a unique identity represented by the flow embeddings and its endpoints. Therefore, information in the flow graph can be propagated by random sampling and iterating identities, regardless of the number of edges between two nodes. In the propagation process, we implement two attention layers, i.e., node-to-edge (n2e) and node-to-node (n2n), to learn the feature of flow-to-flow and flow-to-topology relations. Instead of the scaled dot product, the n2e attention utilizes cosine similarity to measure the similarity between the query and key vectors of the node and the edge, since the cosine similarity can more reflect the pattern similarity rather than the distance between two vectors, which is more suitable to identify the similar flows transferring similar payloads; the n2n attention utilizes euclidean similarity to measure the distance between the query and key vectors, since the node vectors denote the points located in the same euclidean space. At the end of each epoch, the edge embeddings integrate the vectors of both the endpoints. After training, the final edge embeddings, namely, the flow vectors, involve the heterogeneous characteristics of packets, flows, and topology.

In summary, our contributions are as follows.

- We analyze the entropy of more than 22 million packets associated with over one hundred protocols, and identify the minimal number of bytes of the initial packet vectors that should be reserved to guarantee the effectiveness of subsequent flow pretraining. Based on this, we implement a concurrent padding module, which is 12.45 times faster than nPrint [26].
- We convert network flows into fixed-length slices and develop an LSTM-with-attention encoder-decoder to initialize flow embeddings. This model automatically learns sequential (reflecting plaintext protocols with ordered packet content) and nonsequential (capturing ciphertext protocols disrupting plaintext order) packet relation features, enabling SnifferDog to detect both plaintext and ciphertext malicious flows.
- We propose a GNN tailored to flow graphs, which embeds heterogeneous flow-to-flow/flow-to-topology relations via n2e and n2n attention propagation. This allows SnifferDog to capture key features of: 1) multi-step attacks (e.g., RCE, with cooperative attacker-victim flows); 2) distributed attacks (e.g., DDoS, with multi-machine flows targeting a victim).
- We evaluate SnifferDog on 8 datasets collected in different environments, showing it outperforms 6 baselines—especially in multi-class classification—proving superior feature representation. Deployed distributedly in our institute’s network, it exhibits a low false positive rate, verifying real-scenario effectiveness.

¹Source code and datasets: <https://github.com/ciat-gzhu/SnifferDog>

II. BACKGROUND AND MOTIVATION

In this section, we introduce the background knowledge and motivation of this work. First, we briefly discuss the TCP and UDP protocols to specify the heterogeneity of network traffic and standards required by deep learning models. Then, we present an entropy analysis to illustrate the information loss of features extracted from the packet header. Finally, we explain the relations involved in packets, flows, and topology, and explore the motivation of our proposed GNN model.

A. Heterogeneity of Network Traffic

Network protocols define the meaning of bits in packets and describe how messages are exchanged between parties. However, the protocol specifications vary significantly from one another. Taking TCP and UDP, the dominant transferring protocols on the Internet, as examples, the former's header contains at least 20 bytes, defining a 4-byte sequence number, a 4-byte acknowledgment number, etc., to ensure high reliability. In contrast, the UDP header is only 8 bytes long, achieving a high communication speed. A flow corresponds to an entire application layer data transmission, which consists of a sequence of packets. However, the number of packets in a flow is unpredictable. For example, an SSH flow may contain over 10 thousand packets when serving for telecommuting, but may only generate around 16 packets after three failed password attempts.

Deep learning (DL) models are known for their end-to-end learning capabilities. However, the input layer of those DL models typically requires data to be in a normalized format. For example, in the attention mechanism, original input vectors are multiplied by with three matrices W_Q, W_K, W_V to obtain the corresponding Q, K , and V matrices, respectively. The sizes of W_Q, W_K , and W_V are predefined, so the dimension of input vectors must be fixed. While sequence learning models like RNN can process variable-length inputs, their computing speed is significantly slower compared to fixed-length inputs because additional padding and packing processes are introduced, which degrade system performance in real-world applications. Therefore, packets of different sizes cannot be treated as direct input vectors, and flows with varying lengths are not feasible to be processed as sequences for learning. To optimize the throughput, SnifferDog first transforms packets into a format with fixed dimensions and then slices flows into fixed-length segments.

B. Limitations of Using Packet Headers

To investigate the exact information loss of statistical features extracted from the packet headers, we collected one day's traffic from our institute's network, consisting of over 22 million packets with 275 protocols. We then calculated the entropy distributions of their headers and payloads, respectively.

All packets are padded with 0, resulting in a 128-byte header and a 1504-byte payload. The 128-byte header contains a 60-byte IP header, a 60-byte TCP header and an 8-byte UDP header. The entropy is calculated through calculated through formula $E = -\sum_1^{n_p} p(md5_p^i) \log_2 p(md5_p^i)$, where $md5_p^i$ is the MD5 hash code of the header or payload, $count(md5_p^i)$ is the

number of MD5 value i , and n_p is the total number of packets, and $p(md5_p^i) = \frac{count(md5_p^i)}{n_p}$. The results show that the largest entropy of the header and payload is about 24.1 and 17.1, respectively. This suggests the payload contains about 71% of the information present in the header. Using only the packet header for feature extraction results in a loss of about 41% of the packet's information. As mentioned by LeCun et al. [27] in *Nature*, the more the amount of information in the input data, the richer the abstract representations the model can learn, leading to higher accuracy in various tasks. This demonstrates a positive correlation between the amount of information in the input data and the model's accuracy. A 41% information loss significantly undermines the effectiveness and stability of deep learning models.

However, the features of payload cannot be straightforwardly extracted as statistical values, since its bit sequence has no officially defined meaning, unlike the header. Packets in a flow are used to transfer data segments of serialized files, videos, images, and so forth. The payloads of these packets contain semantic relation according to the data they load. In the case of plaintext, the relation is sequential when the data segments are loaded in order, i.e., the sequential relation. In the case of ciphertext, encryption may disrupt the ordered relation. For example, in TLS, the widely implemented encryption algorithm is AES-128 in CMC mode. AES-128 and other symmetric encryption algorithms are all developed upon confusion and diffusion, which disrupt the character order of plaintext. Therefore, the payloads of packets in encrypted traffic involve disrupted semantic relation originating from upper-layer data, i.e. the nonsequential relation. The sequential and nonsequential relations motivate us to develop the flow pretraining component by integrating the LSTM and attention mechanisms.

C. Relations in Flows and Topology

In practice, both benign network activities and attacks tend to generate multiple flows to achieve their objectives. For example, remote login tools like XTerminal [28], which assembles SSH functions, keeps monitoring CPU and memory usage of remote hosts, and generates several long flows. Another benign example is the web browsing, where each request to a website generates multiple HTTP connections to load resources like images, texts, sheets, etc. Each HTTP connection forms a flow, and these flows together form a trace of the website request.

In contrast, malicious activities such as RCE (Remote Code Execution) and DDoS (Distributed Denial of Service) attacks often involve multiple flows that work together to achieve their objectives. Taking the notorious Eternalblue as an example, it is a buffer overflow vulnerability in the Windows Server Message Block (SMB) service that can be exploited to carry out RCE attack. In the reconnaissance stage, an attacker uses probing tools, like nmap, to scan for open ports, confirming that port 445 is exposed. The vulnerability check module then scans the target for the EternalBlue vulnerability, and if successful, a remote control session is established. In the final stage, the attacker can perform malicious actions, such as information gathering, lateral movement & persistence, and data exfiltration & control. All the probing, vulnerability

check, exploit and remote control actions generate flows to maintain a connection to the victim, forming multiple cooperative edges between two nodes in a flow graph. Another example is DDoS attack, in which an attacker organizes a set of compromised devices (often forming a botnet) to disrupt the normal functioning of a targeted server, service, or network by overwhelming it with a flood of internet traffic. This results in a flow-to-topology relation, where flows between different node pairs collaborate to accomplish a distributed attack, forming a star-shaped plot in a flow graph.

GNN is particularly suitable for modeling flow-to-flow and flow-to-topology relations, but most existing GNN models focus on general graphs with a single edge between two nodes. In this work, we attempt to develop a novel GNN model capable of effectively learning features from the heterogeneous relations presented in flow graphs.

III. RELATED WORK

Rule-based IDSs like [29], [30], [31], [32], and [33] drew considerable attention before learning-based approaches became popular, in fields such as DDoS mitigation, phishing detection, and false data injection detection. However, these methods rely on continuous manual updates of the rule library, making them easily evaded by manipulating some key words or snippet of the attack payload or code. Moreover, the rules are usually isolated and cannot capture complex characteristics of attacks. In recent years, learning-based NIDS research has become dominant and can be categorized into TML-based and DL-based. The TML-based models use shallow architectures with one or a few layers (e.g., linear regression, SVMs, decision trees) and rely heavily on manual feature engineering. DL-based works employ deep neural networks with many layers (e.g., CNNs for images, RNNs/Transformers for sequences) and are capable of learning hierarchical features automatically. In the following subsections, we briefly introduce network attacks or anomaly detection studies based on these two models and summarize their limitations.

A. Traditional Machine Learning-Based Approaches

In TML-based approaches, feature engineering aims to logically select a subset of the original features or reduce their dimensionality for use as input to a learning model. This process helps to avoid dimensional disasters, improve generalization performance, and serves as a preliminary step for the classification of NIDS.

Ambusaidi et al. [15] presented a mutual information-based algorithm for automatically selecting optimal features by capturing linear and non-linear correlations. Thaseen and Kumar [14] adopted cardinal feature selection techniques for dimensional reduction. Zhou et al. [19] proposed an intrusion detection framework based on an algorithm called CFS-BA for dimension reduction. Alazzam et al. [16] proposed a wrapper-based feature selection algorithm that used a pigeon-inspired optimizer (PIO) to select optimal feature subsets. Engly et al. [17] used imbalance correction and feature selection techniques to improve the quality of data in intrusion detection. Li et al. [18] identified optimal feature subsets based on random forest with information gain and AP clustering feature grouping algorithms. Barradas et al. [20] introduced FlowLens,

a lightweight system deployed on programmable switches to classify flows. Fu et al. [24] implemented a system, named HyperVision, which identified abnormal flows based on a set of statistical or clustering algorithms.

While the feature selection methods have improved both the accuracy and the computing efficacy of machine learning-based techniques, the heavy dependence on statistical features causes significant information loss. Moreover, these methods rely on heuristic rules or metrics, which limits their ability to learn complex interactions between features.

B. Deep Learning-Based Approaches

Deep learning outperforms traditional machine learning in automatically generating high quality features from raw data. It has demonstrated powerful capabilities in many fields, such as image recognition, document classification and speech recognition, inspiring numerous security researchers to apply deep learning models.

In practice, the statistic features were widely used as original input instead of raw traffic data in DL-based works. Vigneswaran et al. [36] presented a 3-layer DNN to dynamically fuse the statistical features in the KDD-99 dataset. Wang et al. [41] implemented lightweight gradient boosters (LightGBM) to enhance the robustness of the features learned by DNN. Min et al. [10] made use of both word embedding and Text-CNN to extract valid information from the payload. Wang et al. [44] incorporated CNN and long short-term memory (LSTM) to fuse spatial features and temporal features. Wang et al. [39] combined a modified LeNet-5 structure and an LSTM layer to learn spatiotemporal features of flows. Andresini et al. [35] represented network data as a 2D image and a 2D CNN was trained to embed packet streams. Baldoni and Battisti [37] used a few statistics, such as the mean number and the standard deviation of received bytes, and a PCA enhanced histogram-based representation model to detect anomalies in network traffic. Chen et al. [9] developed an NBAD algorithm based on DBN and LSTM to automatically extract features and reduce dimensions. Lo et al. [11] presented a model named E-GraphSage to analyze abnormal topological relations of IoT devices. Akpaku et al. [38] took adversarial actions like adding deceptive positive edges or reducing critical negative edges into account, and presented a robust adaptive GNN to solve this issue. Sayem et al. [5] developed a framework based on LSTM, CNN and GRU, which takes the result of NIDS as input to improve its performance. Fu et al. [43] introduced a deep semantic analysis method based on a U-shaped CNN architecture, taking only the packet length pattern as input to identify tunneled flooding traffic.

Several researches also attempted to develop sophisticated methods to enable raw packet learning or integrate features of heterogeneous relations. For example, Zhang et al. [22] designed TFE-GNN, a model treating bytes as nodes in a graph to represent packets through a self-supervised GNN model. Lin et al. [21] presented a model called ET-BERT that utilizes the Bert model to represent flows based on bigram byte tokens with 65,535 dimensions. These two byte-level methods introduced a significant resource requirement, as a packet typically contains up to 1,500 bytes. Cui et al. [23] utilized Tshark to extract protocol fields in different packet layers, and fed these

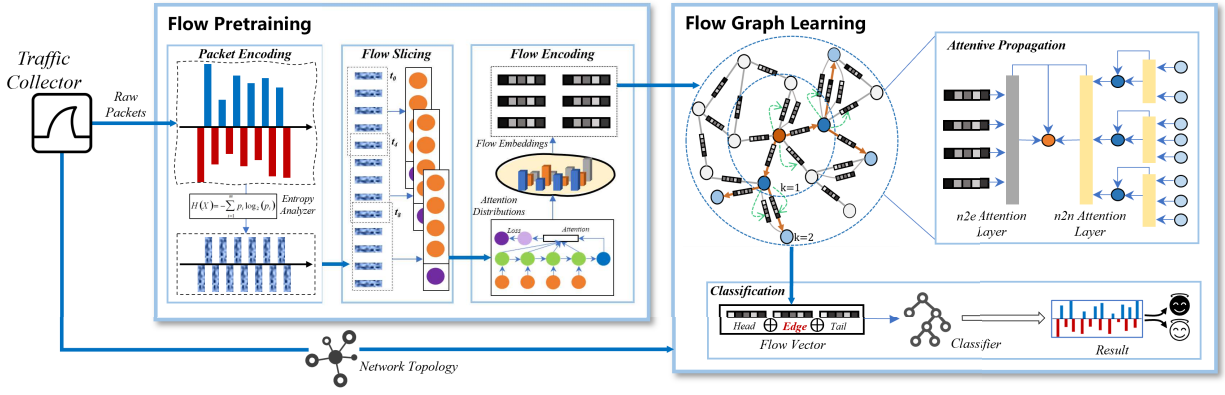


Fig. 1. Overview of SnifferDog.

TABLE I

SUMMARY OF THE INFORMATION USED IN RELATED WORKS. TML: TRADITIONAL MACHINE LEARNING, DL: DEEP LEARNING, P2P: PACKET-TO-PACKET RELATION, F2F: FLOW-TO-FLOW RELATION, F2T: FLOW-TO-TOPOLOGY RELATION

#	Approach	TML	DL	Header	Payload	P2P	F2F	F2T
1	[13]–[20]	✓	✗	✓	✗	✗	✗	✗
2	[24]	✓	✗	✓	✗	✗	✓	✓
3	[8], [34]–[37]	✗	✓	✓	✗	✗	✗	✗
4	[11], [38]	✗	✓	✓	✗	✗	✗	✓
5	[10], [26], [39]	✗	✓	✓	✓	✗	✗	✗
6	[5], [9], [40]–[43]	✗	✓	✓	✗	✓	✗	✗
7	[21]–[23], [40], [44]	✗	✓	✓	✓	✓	✗	✗
8	[12]	✗	✓	✓	✗	✓	✓	✗
9	SnifferDog	✗	✓	✓	✓	✓	✓	✓

fields key-value pairs as language tokens into LLMs. This process required extracting raw packets through Tshark in a structured format, leading to increased resource consumption, and the LLMs are slow and heavy for network traffic analysis. Holland et al. [26] presented a packet padding tool, nPrint, which encodes each packet with a fixed length to facilitate packet classification tasks; however, it analyzes and pads only the packet header. Qu et al. [12] utilized packet size and timestamp as basic information to hierarchically and uniformly embed packet, flow, and trace features of web applications. Although they optimized computational consumption by using only packet size and timestamp, their method showed a low process speed in both training and testing, as since they did not format the length of input flows.

In summary, these approaches have attempted to dynamically generate rich embeddings; however, most fail to efficiently fuse the features from packets, flows, and topology. In contrast to these methods, we implement a highly efficient flow pretraining module that allows batch learning of the packet-to-packet relation. Additionally, we develop a powerful GNN model to represent heterogeneous relations, i.e., the flow-to-flow and flow-to-topology, in a flow graph. Table I presents the features used in existing works, demonstrating that SnifferDog reads the most comprehensive information from network traffic.

IV. METHODOLOGY

Figure 1 provides an overview of SnifferDog, which takes raw packets as input and represents the features possessed in packets, flows and topology to accurately detect malicious

flows. SnifferDog is composed of two critical components: flow pretraining and flow graph learning. In the flow pretraining component, SnifferDog incorporates a flow formatting module designed to efficiently generate uniform inputs, along with a flow encoding module for generating an initial embedding for each flow based on these inputs. In the flow graph learning component, SnifferDog employs our proposed GNN model to learn the flow-to-flow and flow-to-topology relations, and to output the final flow embeddings which are fed into a random forest model to train the classifier. The training tasks for both components are carried out offline to ensure a high real-time efficiency.

A. Flow Pretraining

1) *Flow Formatting*: The objective of Flow Formatting is to standardize raw network packet and flow formats to optimize computational resources and efficiency for subsequent processing stages. This module consists of two core modules: Packet Encoding and Flow Slicing.

Packet Encoding—To standardize the input for subsequent models, SnifferDog fills any empty fields in the packet with 0, just like nPrint [26]. A packet header vector here consists of a 480-bit IP header, a 480-bit TCP header, and a 64-bit UDP header. Unlike nPrint, we exclude ICMP protocol packets from consideration since ICMP cannot form a flow. Additionally, we remove the 64-bit source and destination IP addresses, as well as the 32-bit source and destination ports for the TCP or UDP protocols, from the packet headers. This reduction helps prevent overfitting, ensuring the pretraining process focuses on the information of transferred content, rather than the connection entities, which are better captured at the topology level during the flow graph learning phase. Ultimately, the byte values from the packet headers are used to form a 116-byte vector for each packet. The payload in a packet can exceed 1,500 bytes. After entropy analysis (as discussed in Section II-B), we determine the smoothing point of the packet entropy distribution to be 193 bytes from the original payload. If the payload length is shorter than 193 bytes, it is padded to this length. When combined with the 116-byte header vector, each packet is thus represented by a 309-dimensional vector.

To enhance the runtime performance of the packet encoding process and optimize throughput compared to nPrint, SnifferDog implements several key improvements tailored to real-world environments. First, SnifferDog resolves issues

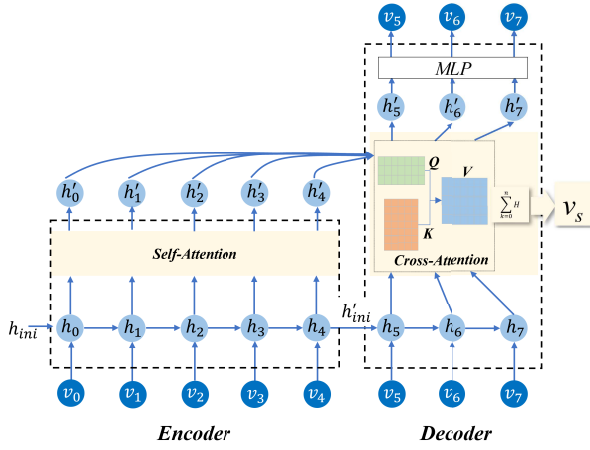


Fig. 2. Example of Flow Encoding.

related to high packet loss rates caused by nPrint's lack of support for VLAN-tagged packets. Second, it increases the original buffer queue size in the libpcap library from 10 MB to 100 MB to support more packet encoding threads (from 2 to 8 threads), thereby enhancing concurrency and reducing latency. Third, SnifferDog identifies memory leaks in nPrint using a valgrind tool [45] and introduces a RAI tool [46] to address the problem. This tool employs a scope guard mechanism, ensuring that memory is dynamically released, which enhances long-term stability during deployment.

Flow Slicing—As mentioned in Section II, a key limitation of LSTM networks and attention mechanisms is their dependence on input samples with consistent time steps. To address this challenge and meet the input requirements of our LSTM and attention-based flow encoding model, we introduce a sliding window mechanism. This mechanism transforms a flow into multiple fixed-length slices, each serving as an encoding unit, ensuring that the flow encoding model can consistently process fixed time steps. Based on the provided quintuples, i.e., source IP, source Port, destination IP, destination Port and protocol, of the packet vectors, SnifferDog aggregates them into a flow sorted by their timestamps. Specifically, for the packet sequence $p_i^1, p_i^2, \dots, p_i^{n_i}$ of the i -th flow $flow_i$ (assuming that $flow_i$ has n_i packets), each packet is encoded and uniformly transformed into a 309-dimensional vector v_i^k ($1 \leq k \leq n_i$). Next, a sliding window sequence is constructed with a window size l_w and a step size of l_s for vector sequences $v_i^1, v_i^2, \dots, v_i^{n_i}$ ($v_i \in R^{309 \times 1}$), namely $[v_i^0, \dots, v_i^{l_w-1}]$, $[v_i^{l_s}, \dots, v_i^{l_w+l_s-1}]$, $\dots$, $[v_i^{n_i-l_w}, \dots, v_i^{n_i-1}]$.

2) **Flow Encoding**: Our flow encoding model consists of two main components: an Encoder and a Decoder, as illustrated in Figure 2. The Encoder incorporates an LSTM network to encode first several packets of a slice as input and a self-attention layer to share information with each other packets. The Decoder includes an LSTM network and a cross-attention layer. It takes the last few packets of a slice, the output hidden vectors of the encoder LSTM and self-attention layers, as input, and generates the slice embedding v_s . The sequential relation is represented by the LSTM layer, while the feature of nonsequential relation is captured by the self-attention and cross-attention layers. SnifferDog aggregates the embeddings of the slices belonging to the same flow by averaging them, which serve as the initial embedding for

the flow. It is important to note that, unlike the transformer decoder which processes elements of a sequence one by one, our decoder reads all the packet vectors simultaneously to accelerate the process. This approach aligns with our goal of capturing relational features and fusing them with the packet vectors, rather than performing content generation tasks, as is typical with Transformer models.

During the training phase, the Decoder's output is compared with the input vectors to minimize the mean squared error (MSE) loss. The hidden state $h_i \in R^{128 \times 1}$ at the i -th time step in the LSTM layer can be calculated as:

$$z_i = \sigma(\mu_z v_i + W_z h_{i-1} + b_z) \quad (1)$$

$$f_i = \sigma(\mu_f v_i + W_f h_{i-1} + b_f) \quad (2)$$

$$o_i = \sigma(\mu_o v_i + W_o h_{i-1} + b_o) \quad (3)$$

$$c_i = \tanh(\mu_c v_i + W_c h_{i-1} + b_c) \quad (4)$$

$$c_i = f_i * c_{i-1} + z_i * c_i \quad (5)$$

$$h_i = \tanh(c_i) * o_i \quad (6)$$

Specifically, $\mu_j \in R^{128 \times 309}$, $W_j \in R^{128 \times 128}$ and $b_j \in R^{128 \times 1}$ ($j \in \{z, f, o, c\}$) are learnable parameters. Here, z_i, f_i, o_i , and c_i are the input gate, forget gate, output gate, and the i -th content vector, respectively. σ is the logistic sigmoid function, $\tanh$ is the $\tanh$ activation function, and $*$ is the Hadamard product. Assuming that the number of packets in the sliding window is l_w , the first l_e packet vectors are input into Encoder's LSTM network, that is, the input sequence length is l_e , and then the rest l_d vectors are fed to the Decoder ($l_d = l_w - l_e$). This model compresses a slice of high-dimensional ($309 \times l_w$) packet features into a lower-dimensional (128) embedding.

The self-attention layer takes the matrix H_{self} consisting of the l_e hidden $h_0, \dots, h_{l_e-1}$ as input, where $H_{self} = [h_0, \dots, h_{l_e-1}]$. The Q_{self}, K_{self} and V_{self} matrices of the self-attention layer are calculated as follows:

$$Q_{self} = H_{self} W_q^s \quad (7)$$

$$K_{self} = V_{self} = H_{self} W_{kv}^s \quad (8)$$

where $W_q^s, W_{kv}^s \in R^{128 \times 128}$ are the trainable parameters. The output matrix H'_{self} is computed using the Scaled Dot-Product Attention:

$$H'_{self} = \text{softmax} \left(\frac{Q_{self} K_{self}^T}{\sqrt{d_s}} \right) V_{self} \quad (9)$$

where d_s is the dimension of the hidden vectors (i.e., 128 in this work), and $H'_{self} = [h'_0, \dots, h'_{l_e-1}]$ serves as an input to the cross-attention layer of the decoder.

The input of the decoder consists of three parts: the rest packet vectors $v_{l_e}, \dots, v_{l_w-1}$, the output hidden vector h'_{ini} of the Encoder's LSTM layer, and H'_{self} . The LSTM layer of the Decoder operates similarly to the Encoder's LSTM and generates $H_{cross} = [h_{l_e}, \dots, h_{l_w-1}]$. The Q_{cross}, K_{cross} and V_{cross} matrices of the cross-attention layer are calculated as follows:

$$Q_{cross} = H_{cross} W_q^c \quad (10)$$

$$K_{cross} = V_{cross} = H'_{self} W_{kv}^c \quad (11)$$

where $W_q^c, W_{kv}^c \in R^{128 \times 128}$ are the trainable parameters. The output matrix H'_{cross} is computed as the Scaled Dot-Product Attention:

$$H'_{cross} = softmax\left(\frac{Q_{cross}K_{cross}^T}{\sqrt{d_c}}\right)V_{cross} \quad (12)$$

where d_c is the dimension of the hidden vectors (i.e., 128 in this work), and $H'_{cross} = [h'_{l_e}, \dots, h'_{l_w-1}]$. After process by a two-layer Multilayer Perceptron (MLP), we have the output matrix $Z = z_{l_e}, \dots, z_{l_w-1}$ of the decoder. The loss function is computed as follows:

$$Loss_{fp} = \frac{1}{l_w - l_e} \sum_{z_i} \|z_i - v_i\|_2 \quad (13)$$

After training, the optimized parameters are obtained, and a slice embedding is calculated by aggregating the vectors in H'_{cross} , i.e., $v_s = \frac{1}{l_w - l_e} \sum_{h_i}^{H_{cross}} h_i$.

As shown in Figure 2 by an example, assuming $l_w = 8$ and $l_e = 5$, there is a vector sequence $[v_0, \dots, v_7]$. The LSTM network of the Encoder calculates the input sequence $[v_0, \dots, v_4]$ to obtain the corresponding hidden state vectors $[h_0, \dots, h_4]$. The hidden state h_4 is fed into the Decoder for initialization. The self-attention layer outputs $H_{self} = [h'_0, \dots, h'_4]$. The LSTM layer of the Decoder produces $H_{cross} = [h_5, h_6, h_7]$, and the cross-attention layer takes both H_{self} and H_{cross} as input to compute $H'_{cross} = [h'_5, h'_6, h'_7]$. After training, the slice embedding is computed as $v_s = \frac{h'_5 + h'_6 + h'_7}{3}$. During a real-time encoding scenario, after flow formatting, the flow is segmented into a series of slices: $slice_0 = [v_i^0, \dots, v_i^{l_w-1}]$, $slice_1 = [v_i^{l_s}, \dots, v_i^{l_s+l_w-1}]$, ..., $slice_{\left(\frac{n_i-l_w+l_s-1}{l_s}\right)} = [v_i^{n_i-l_w}, \dots, v_i^{n_i-1}]$. For each slice, based on the pretrained parameters, we have the representation vector v_s for that slice. At last, we aggregate the average of representation vectors from all slices in the flow to obtain the initial flow embedding $v_f = \frac{1}{n_i-l_w+l_s-1} \sum v_s$, where $v_f \in R^{128 \times 1}$.

B. Flow Graph Learning

Based on the connectivity of network devices, we construct a flow graph where IP addresses serve as nodes, and flows between IP addresses serve as edges connecting the nodes. The results of the flow encoding model are used as initial embeddings for the flows, i.e., the initial attributes of the edges. This results in an attributed flow graph. In the flow graph, multiple edges may exist between two IP addresses, and simply aggregating them as one cannot represent their collaborating relationships. Therefore, the flow graph is represented as $AIn_{fa} = \{V, E\}$, where $v \in V$ is an IP address and an edge $e \in E$ is identified as a triple $(src, feat, dst)$. Here, src , dst , and $feat$ represent the source IP address, destination IP address, and flow vector, respectively.

In message propagation, random edge and node sampling are performed on the flow graph respectively to obtain the propagating candidates and to reduce computing complexity. In terms of graph AIn_{fa} , let $N(v)$ be all sampled nodes among the neighboring nodes of node v (hereinafter referred to as the neighboring nodes of node v), and $D(v)$ be all sampled edges among the edges connected to node v (hereinafter referred to

as the edges connected to node v). For node v , the message propagation is performed recursively as follows:

$$h_v^{(k)} = \sigma(g^v(h_v^{(k-1)}, h_{N(v)}^{(k-1)}, h_{D(v)}^{(k-1)})W^{(k)} + b^{(k)}) \quad (14)$$

where $h_v^{(k)}$ is the feature vector of node v after propagation in the k -th layer; $\sigma(\cdot)$ represents a differentiable non-linear activation function, with *LeakyReLU* used here; $g^v(\cdot)$ is the aggregation function, denoted as a SUM operation; $h_{N(v)}^{(k-1)}$ is the aggregated representation of all neighboring nodes $N(v)$ of node v at the $(k-1)$ -th layer; and $h_{D(v)}^{(k-1)}$ represents the aggregated representation of all connected edges $D(v)$ of node v at the $(k-1)$ -th layer. When $k = 0$, $h_v^{(0)} = W_{ini}b_{ip}$, where $W_{ini} \in R^{128 \times 32}$ is a learnable weight matrix, and b_{ip} is the bit sequence of IP address of the node; and $W^{(k)}$ and $b^{(k)}$ are the learnable weight matrix and bias vector at the k -th layer, respectively. From the formula above, it is evident that the key to computing the node embedding h_v is to aggregate the features of its neighbors $h_{N(v)}$ and edges $h_{D(v)}$.

1) *The n2e and n2n Attentions*: As shown in the right part of Figure 1, the n2e and n2n attentions are used to aggregate the neighboring node and edge features, respectively. In the n2e attention layer, the cosine similarity is employed to calculate the attention score, since it focuses on capturing similar patterns rather than quantitative values of vectors. The pattern here resembles a density distribution of the dimensions of a flow embedding, which reflects both the packet contents and the relations between them. Cosine similarity is suitable for measuring the patterns since it removes the impact of ranges by dividing lengths of the vectors. Specifically, if a payload is transferred multiple times in a flow and only once in another, the cosine similarity effectively captures the content similarity. This is exactly the purpose for identifying similar or correlated malicious content across different flows. The n2e attention acts as an intermediary to facilitate message propagation from one edge to another. We implement this intermediary propagation because the number of nodes in the flow graph is much smaller than that of edges, reducing the computing complexity from $O(|E|^2)$ to $O(|V| * |E|)$. Euclidean similarity is employed to measure the relation between two nodes in n2n attention layer. The reason is that the nodes represent IP addresses which are pretty suitable to be measured the distance by euclidean distance rather than dot product-based measurement like the cosine similarity. For example, the similarity between IP addresses 1.1.1.1 and 2.2.2.2 is calculated to be 1 by computing the cosine similarity, while they are distributed in different class A addresses, and only 0.33 by computing the euclidean similarity. When an attack launched multiple times, the attention weight (i.e., the similarity) becomes large between these flows and nodes generated by this attack, so the flow vectors become close. This is similar for benign applications, of which the flow vectors become close during training phase. The differentiated flow vectors facilitate classifying attack traffic.

For the n2e attention, the attention score $s(v, e)$ for the node v and the edge e ($e \in D(v)$) is calculated as follows:

$$s(v, e) = Dist_{cos}(h_v W_e, r_e) \quad (15)$$

where h_v is the feature vector of node v , W_e is the learnable weight matrix used to map the node feature h_v to the edge

feature r_e , and $Dist_{cos}()$ is the cosine similarity function. After calculating the attention scores $s(v, e)$ for all edges e in $D(v)$, the aggregated edge information $h_{D(v)}$, i.e., the result of the n2e attention weighted for node v , is described as follows:

$$\alpha_e = \frac{\exp(s(v, e))}{\sum_{e' \in D(v)} \exp(s(v, e'))} \quad (16)$$

$$Attn_{D(v)} = \{\alpha_e, e \in D(v)\} \quad (17)$$

$$h_{D(v)} = (\sum_{e \in D(v), \alpha_e \in Attn_{D(v)}} \alpha_e r_e) W_v \quad (18)$$

where $\exp()$ is the exponential function, r_e is the attributes of the edge, and a_e is the attention value corresponding to the edge. $Attn_{D(v)}$ is the attention distribution over all edges, and W_v is the learnable weight matrix that maps the aggregated edge information back to the node features.

The attention score between node v and its neighboring node u (where $u \in N(v)$) is calculated as follows:

$$s(v, u) = Dist_{euc}(h_v, h_u) \quad (19)$$

where $Dist_{euc}()$ is the Euclidean similarity function. Then, the node information aggregation result $h_{N(v)}$ after attention weighted aggregation of node messages is obtained as follows:

$$\alpha_u = \frac{\exp(s(v, u))}{\sum_{u' \in N(v)} \exp(s(v, u'))} \quad (20)$$

$$Attn_{N(v)} = \{\alpha_u, u \in N(v)\} \quad (21)$$

$$h_{N(v)} = \sum_{u \in N(v), \alpha_u \in Attn_{N(v)}} \alpha_u h_u \quad (22)$$

Here, α_u is the attention value corresponding to node u , and $Attn_{N(v)}$ is the attention distribution between node v and its neighboring nodes.

Algorithm 1 Topology Feature Learning

Input: $AIN_{fa} = \{V, E\}$;

- 1: Node Features $h_v^0, \forall v \in V$;
- 2: Edge Features $r_e, \forall e \in E$;
- 3: message propagation Layers K ;
- 4: Weight Matrices W_e, W_v ;
- 5: non-linearity σ ;
- 6: Differentiable Aggregator Functions g^v, g^e ;

Output: Edge Embedding $r'_e, \forall e \in E$

```

7: for  $k \leftarrow 1$  to  $K$  do
8:   for  $v \in V$  do
9:      $s(v, e) \leftarrow Dist_{cos}(W_e h_v^{(k-1)}, r_e^{(k-1)}), \forall e \in D(v)$ 
10:     $\alpha_e = \frac{\exp(s(v, e))}{\sum_{e' \in D(v)} \exp(s(v, e'))}$ 
11:     $Attn_{D(v)} = \{\alpha_e, e \in D(v)\}$ 
12:     $h_{D(v)}^{(k-1)} \leftarrow (\sum_{e \in D(v), \alpha_e \in Attn_{D(v)}} \alpha_e r_e) W_v, \forall e \in D(v)$ 
13:     $s(v, u) \leftarrow Dist_{euc}(h_v^{(k-1)}, h_u^{(k-1)}), \forall u \in N(v)$ 
14:     $\alpha_u = \frac{\exp(s(v, u))}{\sum_{u' \in N(v)} \exp(s(v, u'))}$ 
15:     $Attn_{N(v)} = \{\alpha_u, u \in N(v)\}$ 
16:     $h_{N(v)}^{(k-1)} \leftarrow \sum_{u \in N(v), \alpha_u \in Attn_{N(v)}} \alpha_u h_u, \forall u \in N(v)$ 
17:     $h_v^{(k)} \leftarrow \sigma(g^v(h_i^{(k-1)}, h_{N(i)}^{(k-1)}, h_{D(i)}^{(k-1)})) W^{(k)} + b^{(k)}$ 
18:   end for
19: end for
20: for  $e \in E$  do
21:    $r'_e \leftarrow g^e(h_{head}^{(k)}, r_e, h_{tail}^{(k)})$ 
22: end for

```

With the message propagation completed, the final vector of the edge is updated, and the process is formulated as follows:

$$r'_{(u,f,v)} = g^e(h_v^{(k)}, r_{(u,f,v)}, h_u^{(k)}) \quad (23)$$

where $g^e()$ represents the aggregation operation. The CONCAT operation is employed to aggregate edge features with node features. Analysis of the impact of the number of message propagation layers k (from 1 to 3) on our GNN model reveals that the optimal performance is generally achieved when k is set to 2. The possible reason is that these datasets contain rare complex attacks that involves multiple IP addresses. The primary objective of our work is to enhance the efficacy of identifying malicious network flows, rather than establishing correlations across all attack steps. Therefore, in our system, the value of k is set to 2, which reduces the resource consumption associated with the propagation process. The complete algorithmic process is detailed in Algorithm 1. Unlike methods such as Graph Convolutional Networks (GCN) that rely on pre-defined topological parameters (e.g., adjacency matrices), our approach requires no such prior restriction. Consequently, it enables inductive learning, like GraphSage [47] and Jbeil [48], with uncertain node distributions—a critical advantage for handling dynamic or incompletely defined graph structures.

2) *Classification:* We choose Cross Entropy Loss and mini-batch ADAM optimizer to optimize the topology embedding model. The early stopping strategy is chosen for the training task. Specifically, $Z = [z_0, z_1, \dots, z_{C-1}]$ denotes the non-softmax output of a flow, where C is the number of labels. Then, the loss function equation can be expressed as:

$$\begin{aligned} loss(z, c) &= -\log \left(\frac{\exp(z[c])}{\sum_{j=0}^{C-1} \exp(z[j])} \right) \\ &= -z[c] + \log \left(\sum_{j=0}^{C-1} \exp(z[j]) \right) \end{aligned} \quad (24)$$

where c is the label of a specific sample. During the training phase, flow vectors with the same label gradually share large attention weights than others. This occurs because the payloads transferred through these flows either contain similar content, such as in DDoS and Brute attacks, or exhibit similar patterns when the same multi-step attacks are repeated. As a result, during testing or real-time classification, the vectors of flows serving for the same attacks are closer than that for others, ensuring high detection effectiveness.

V. EVALUATION

To evaluate SnifferDog, we tested its performance on eight network traffic traces collected from different environments like IoT, campus network, VPN, etc. Seven of them are open-source, ensuring reproducibility of our experiments, and one was generated by ourselves. Moreover, we discuss the model's real-world deployment in our institute's network. The model training and testing are based on a machine with an Intel Xeon Gold5318Y@2.10GHz 24Cores x2 CPU, an NVIDIA A800 GPU card, and a 256GB memory. The training and testing datasets were divided in a 7:3 ratio for all subsequent experiments.

TABLE II
OVERVIEW OF THE DATASETS USED IN OUR EXPERIMENTS

#	Dataset	Host	Flow	Normal(%)	Attack Type	Format
1	UNSW-NB15	686,544	1,097,850	70.73%	9	CSV
2	ToN-IoT	353,481	1,000,499	78.82%	10	CSV
3	CICIDS-2017	229,976	973,319	82.50%	6	PCAP
4	Darknet-2020	57,931	141,530	82.80%	9	CSV
5	Bot-IoT	261,190	806,936	0.05%	4	CSV
6	CICDDoS-2019	130,079	888,825	0.93%	10	CSV
7	ISCX-2012	112,897	229,655	93.5%	4	PCAP
8	Customized	23,261	1,813,095	64.3%	18	PCAP

A. Dataset

The eight datasets used in the evaluation, including UNSW-NB15 [49], ToN-IoT [50], Darknet-2020 [51], Bot-IoT [52], CICDDoS-2019 [53], CICIDS-2017 [54], ISCX-2012 [55], and a customized one generated by us, are summarized in Table II. Among them, only the CICIDS-2017, ISCX-2012, and customized dataset provide pcap files. These three datasets were used to evaluate the entire process of SnifferDog, while the others were employed to assess the effectiveness of the flow graph learning component. We used the CICFlowMeter [56] to extract features from pcap files for the baseline methods. The details of these datasets are as below.

- **UNSW-NB15** is generated by the IXIA PerfectStorm tool [57]. The Argus [58] and Bro-IDS tools [59] are used to generate 49 features for each flow.
- **ToN-IoT** consists of network traffic of new generations of IoT/IIoT, and has been widely used to evaluate AI-enabled cybersecurity applications.
- **CICIDS-2017** contains network traffic in packet-based and bidirectional stream formats that are collected in a simulated environment over one week.
- **Darknet-2020** is a dark network dataset created by merging two public datasets, ISCXTor2016 and ISCXVPN2016, to detect and characterize VPN and Tor applications.
- **Bot-IoT** is a dataset created by the University of Canberra's Cyber Range Lab in New South Wales. This dataset consists of attack activities in IoT scenarios.
- **CICDDoS-2019** contains both benign flows and the most latest common DDoS attacks. The B-Profile system [60] is used to build benign behaviors.
- **ISCX-2012** consists of 7 days of network activity, including full packet payloads in pcap format, along with the relevant profiles. The packet payloads involve 4 attacks.
- **Customized** is a dataset collected in an institute network environment (detailed in Section V-F) from October 20th to 26th, 2023. It contains 18 types of attacks, including dos, brute, backdoor, remote code execution and so forth. Most of these attacks are generated using tools from Kali system except PHPmyadmin Brute (No. 14), Web Shell (No. 15) and Drupal RCE (No. 16). Targets are randomly selected, ranging from 20 to 100 addresses (less than 0.5%) for each attack, and source addresses are random selected ranging from 2 to 10 (less than 0.05%). The parameters for reproducing these attack traffic data are provided in Table III to facilitate the reproduction of these traffics. Notably, a large portion (35.7%) of the generated attack traffic was specifically designed to

TABLE III
DETAILS OF OUR ATTACK DATASET

#	Attack Type	Flows	File Size	Tool	Duration
1	SYN DoS	15,552	396MB	LOIC	1 hour
2	TCP DoS	155,952	1.4GB		2 hours
3	UDP DoS	76,350	11GB		5 hours
4	Ssdp Flood	78,950	629MB		1 hour
5	HTTP DoS	73,545	673MB		2 hours
6	SSH Brute	22,945	237MB	Patator	2 days
7	FTP Brute	23,269	181MB		2 days
8	Mysql Brute	14,586	94MB		1 day
9	Telnet Brute	49,522	443MB		3 days
10	SMB Brute	14,107	265MB		2 days
11	Fuzzing	22,463	470MB		4 hours
12	RDP Brute	8,050	62MB	Hydra	1 day
13	Wordpress Brute	7,975	44MB	Wpscan	1 day
14	PHPmyadmin Brute	3,241	297MB	PHPAdmin Brute1.3	1 day
15	Web Shell	31,178	143MB	Ice Scorpion 4.0	6 hours
16	Drupal RCE	13,837	114MB	Author Customized Tools	3 hours
17	ARP MitM	21,239	453MB	ARPSpoof	4 hours
18	SSL Recognition	14,090	391MB	The-ssl-dos	4 hours

measure the multi-class classification effectiveness of SnifferDog, even though attack traffic is relatively rare in real-world scenarios.

Though four of the five CSV-format datasets (excluding Darknet-2020) provide pcap files, the profiles and labels they provide cannot be correctly mapped to the flows in their pcap files. Therefore, we only use the CSV records to avoid errors.

B. Comparative Experiment

Accuracy, precision, recall, and F1 score are used as the performance indicators in this paper. Accuracy evaluates the overall correctness on both normal and abnormal flows. Precision measures the ability to correctly identify attack flows, while recall assesses the ability to detect all attacks. F1 score is the harmonic mean of precision and recall, which tests the overall effectiveness of models. We conducted a comparison test involving several classifiers, including multilayer perceptron (MLP), SVM, and random forest [18], [61], [62]. The results indicate outperforms the others, achieving a performance improvement of approximately 2.4% over MLP and 3.3% over SVM on our dataset, as introduced in Section V-A, respectively. Therefore, SnifferDog utilizes random forest as the final classifier.

1) *Baselines*: We conduct both binary-class and multi-class experiments upon the eight datasets in comparison to six baselines, including two DNN-based [36], [41], a RNN-based [34], a GNN-based [11] models, a recent hierarchical deep learning framework presented by Qu et al. [12], and a machine learning based anomaly detection system proposed by Fu et al. [24]. The first four methods are widely used deep learning models (CNN, DNN, LSTM and GNN), all of which rely on statistical features and can be applied to all the eight datasets. They can be used to compare the effectiveness of our flow encoding and proposed GNN models. The last two methods are SOTA works that implement an assembled structure to identify malicious flows and use pcap files as input. They therefore only provide results for the three pcap datasets. Table VI presents the parameters involved in the experiments, which were manually and meticulously tuned to achieve optimal performance across the above mentioned datasets.

TABLE IV

BINARY-CLASS RESULT.A: ACCURACY,P: PRECISION, R: RECALL,F1: F1 SCORE.THE BEST METRICS ARE HIGHLIGHTED IN BOLD

Dataset	Model	A	P	R	F1
UNSW-NB15	DNN3	0.9908002	0.990943305	0.9908002	0.990827679
	EFS-DNN	0.990903405	0.991077479	0.990903405	0.990934228
	CNN+RNN	0.990496577	0.990679192	0.990496577	0.990529421
	E-GraphSAGE	0.990645833	0.99087284	0.990645833	0.99068267
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.993663372	0.993708673	0.993663372	0.993673541
ToN-IOT	DNN3	0.956734966	0.958510245	0.956734966	0.957295065
	EFS-DNN	0.850721306	0.861670624	0.850721306	0.82024621
	CNN+RNN	0.90691321	0.906815296	0.90691321	0.906863794
	E-GraphSAGE	0.994468571	0.994462203	0.994468571	0.994464082
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.99942029	0.999420211	0.99942029	0.999420226
CICIDS-2017	DNN3	0.99574083	0.995728553	0.99574083	0.99573099
	EFS-DNN	0.994425264	0.99440312	0.994425264	0.994400335
	CNN+RNN	0.996791086	0.996794475	0.996791086	0.996774543
	E-GraphSAGE	0.9983625	0.99836159	0.9983625	0.998359173
	Fu et al. [24]	0.898148761	0.470572624	0.475834548	0.473134429
	Qu et al. [12]	0.93318344	0.93890748	0.93318344	0.932964607
	Ours	0.999789222	0.9997892	0.999789222	0.999789208
Darknet-2020	DNN3	0.957982995	0.958499671	0.957982995	0.958207614
	EFS-DNN	0.937396957	0.936131692	0.937396957	0.936600155
	CNN+RNN	0.95593396	0.955117518	0.95593396	0.955267879
	E-GraphSAGE	0.980333333	0.980200391	0.980333333	0.980109133
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.98895405	0.988912587	0.98895405	0.98888763
Bot-IoT	DNN3	0.999388634	0.999389008	0.999388634	0.999083045
	EFS-DNN	0.99980998	0.999816196	0.99980998	0.999812603
	CNN+RNN	0.999971084	0.999970889	0.999971084	0.999970915
	E-GraphSAGE	0.999929078	0.999929083	0.999929078	0.999926835
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	1.000000000	1.000000000	1.000000000	1.000000000
CICDDoS-2019	DNN3	0.999838739	0.999839401	0.999838739	0.999838984
	EFS-DNN	0.999737482	0.99973974	0.999737482	0.999738272
	CNN+RNN	0.999677477	0.99968335	0.999677477	0.999679262
	E-GraphSAGE	0.999935484	0.999935449	0.999935484	0.999935462
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.999973748	0.999973793	0.999973748	0.999973761
ISCX-2012	DNN3	0.994411948	0.994368304	0.994411948	0.994338486
	EFS-DNN	0.994803762	0.994785148	0.994803762	0.994726087
	CNN+RNN	0.99510864	0.995096591	0.99510864	0.995035542
	E-GraphSAGE	0.991	0.991049738	0.991	0.990702083
	Fu et al. [24]	0.936921876	0.469133405	0.499122238	0.482804702
	Qu et al. [12]	0.885882353	0.89890519	0.885882353	0.832276247
	Ours	0.998795303	0.998795127	0.998795303	0.998790812
Customized	DNN3	0.999753135	0.999753136	0.999753135	0.999753133
	EFS-DNN	0.999795283	0.999795282	0.999795283	0.999795283
	CNN+RNN	0.999530355	0.999530429	0.999530355	0.999530369
	E-GraphSAGE	0.998414508	0.998418989	0.998414508	0.998414905
	Fu et al. [24]	0.975548436	0.452315038	0.476103433	0.463911052
	Qu et al. [12]	0.99315749	0.98434395	0.97399786	0.99313463
	Ours	0.999982287	0.999982287	0.999982287	0.999982287

2) *Binary Classification*: Table IV presents the binary classification results for SnifferDog and six baseline methods. SnifferDog demonstrates excellent performance across all datasets, with all evaluation metrics exceeding 0.98, and outperforms the other models. On the Bot-IoT dataset, SnifferDog achieves performance, with all metrics reaching a score of 1, indicating accurate identification of malicious flows. On the ToN-IoT dataset, SnifferDog's metrics are approximately 0.49% higher than the second-best method, i.e., E-GraphSAGE, and more than 4% higher than other methods, showing a significant improvements across all metrics. Compared to the top four baseline models based on statistical features, SnifferDog yields better results, which highlights the effectiveness of our flow graph learning component. Qu et al.'s model achieves over 99% on the metrics on our dataset, but it falls below 90% on the ISCX-2012 dataset, indicating unstable effectiveness across different environments. In contrast,

TABLE V

MULTI-CLASS RESULT.A: ACCURACY,P: PRECISION, R: RECALL,F1: F1 SCORE. THE BEST METRICS ARE HIGHLIGHTED IN BOLD

Dataset	Model	A	P	R	F1
UNSW-NB15	DNN3	0.960210108	0.960623948	0.960210108	0.956559145
	EFS-DNN	0.955901553	0.958774201	0.955901553	0.953383113
	CNN+RNN	0.957389443	0.95794285	0.957389443	0.95349479
	E-GraphSAGE	0.957739583	0.961436509	0.957739583	0.955035896
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.965207755	0.966269288	0.965207755	0.964228022
ToN-IOT	DNN3	0.926170248	0.934556155	0.926170248	0.924854673
	EFS-DNN	0.789298684	0.785508872	0.789298684	0.808945262
	CNN+RNN	0.858137598	0.85533068	0.858137598	0.817544668
	E-GraphSAGE	0.985817143	0.986296156	0.985817143	0.985760509
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.998752288	0.998755291	0.998752288	0.998748997
CICIDS-2017	DNN3	0.996874671	0.996870113	0.996874671	0.996429519
	EFS-DNN	0.894189047	0.880126955	0.894189047	0.846835666
	CNN+RNN	0.995181178	0.995011251	0.995181178	0.994534119
	E-GraphSAGE	0.9975125	0.997436955	0.9975125	0.997368201
	Fu et al. [24]	0.871835105	0.608159249	0.508121872	0.500561409
	Qu et al. [12]	0.909902474	0.915453534	0.909902474	0.910325522
	Ours	0.998208387	0.998212923	0.998208387	0.998179056
Darknet-2020	DNN3	0.947360984	0.946520372	0.947360984	0.945474501
	EFS-DNN	0.931979839	0.928479605	0.931979839	0.926130556
	CNN+RNN	0.941331638	0.940387715	0.941331638	0.93729758
	E-GraphSAGE	0.972416667	0.971237173	0.972416667	0.970556878
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.984502697	0.983682762	0.984502697	0.983555094
Bot-IoT	DNN3	0.99951256	0.999512554	0.99951256	0.999511642
	EFS-DNN	0.888838401	0.900596599	0.888838401	0.882655305
	CNN+RNN	0.96242993	0.965551357	0.96242993	0.962477968
	E-GraphSAGE	0.902255319	0.902216712	0.902255319	0.902233265
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.999983477	0.999983477	0.999983477	0.999983477
CICDDoS-2019	DNN3	0.752917704	0.702035973	0.752917704	0.773956581
	EFS-DNN	0.727700939	0.68157797	0.727700939	0.677009354
	CNN+RNN	0.738104167	0.770065609	0.738104167	0.685031514
	E-GraphSAGE	0.443819355	0.464678579	0.443819355	0.429201044
	Fu et al. [24]	-	-	-	-
	Qu et al. [12]	-	-	-	-
	Ours	0.793319282	0.781882498	0.793319282	0.779317132
ISCX-2012	DNN3	0.989564132	0.989355833	0.989564132	0.988459894
	EFS-DNN	0.987517418	0.987855137	0.987517418	0.986073242
	CNN+RNN	0.989825392	0.989598501	0.989825392	0.988844843
	E-GraphSAGE	0.9855	0.984740184	0.9855	0.983579061
	Fu et al. [24]	0.898170453	0.470428348	0.475818109	0.473124857
	Qu et al. [12]	0.885882353	0.89890519	0.885882353	0.832276247
	Ours	0.99864673	0.998640924	0.99864673	0.998588369
Customized	DNN3	0.97582162	0.97183221	0.97971488	0.97548655
	EFS-DNN	0.96539456	0.95687099	0.94899456	0.95283318
	CNN+RNN	0.85355488	0.85631917	0.84585488	0.85174244
	E-GraphSAGE	0.89656716	0.90189811	0.89876716	0.90028419
	Fu et al. [24]	0.88161309	0.81154414	0.85483093	0.83264747
	Qu et al. [12]	0.87762097	0.44887972	0.50312097	0.47424734
	Ours	0.99660342	0.98946352	0.99834638	0.99567577

SnifferDog consistently exhibits stable performance across multiple datasets, achieving over 98%, and demonstrating superior generalization capabilities.

3) *Multi-Class Classification*: The results for multi-class classification are presented in Table V. It can be observed that SnifferDog outperforms the other models across all datasets on every metric. Notably, on the CICDDoS-2019 dataset, SnifferDog's accuracy is approximately 4%, 6.6%, 5.5%, and 35% higher than that of the other methods. For the ToN-IoT dataset, SnifferDog's F1 score is respectively about 7.3%, 18.9%, 18.1%, and 1.2% higher than the others. Moreover, SnifferDog exhibits more stable performance across different datasets, as shown by the pink curve in Figure 3. This stability reflects SnifferDog's strong generalization capability and adaptability in heterogeneous network environments. It also highlights its comprehensive feature learning ability, confirming that our method has the minimal information loss. The

TABLE VI
PARAMETER SETTINGS

#	Parameter	SnifferDog	E-GraphSAGE	DNN3	EFS-DNN	CNN+RNN	Qu et al. [12]	Fu et al. [24]
1	Learning rate	7e-3	1e-3	1e-2	1e-4	1e-3	1e-3	-
2	Weight decay	0	0	0	0	0	0	-
3	Optimizer	Adam	Adam	Adam	Adam	Adam	AdamW	-
4	Batch size	2048	2048	1024	1024	1024	128	-
5	Dropout	0.2	0.2	0.01	0.1	0.6	0.5	-
6	GNN layers	2	2	-	-	-	-	-
7	Epoch	1	1	20	20	20	100	-
8	Number of samples	200	200	-	-	-	-	-
9	Activation function	LeakyReLU	ReLU	ReLU	LeakyReLU	ReLU	LeakyReLU	-
10	Hidden layer dimension	128	128	768	512	128	128	-
11	Neighborhood radius threshold	-	-	-	-	-	-	4×10^{-3}
12	Minpoint	-	-	-	-	-	-	40
13	K-means hyper-parameter	-	-	-	-	-	-	10

TABLE VII

THE MULTI-CLASS RESULT OF FPR, MCC AND DETECTION LATENCY (D_{la}). HERE ONLY PRESENTS THE RESULT OF SNIFFERDOG, COMPARING WITH THE BEST PERFORMANCE OF THE BASELINES INSIDE THE BRACKETS, FOR THE SAKE OF BREVITY

Metric	UNSW-NB15	ToN-IoT	CICIDS-2017	Darknet-2020	Bot-IoT	CICDDoS-2019	ISCX-2012	Customized
MCC	0.9256 (0.0123 $\uparrow$)	0.9949 (0.0331 $\uparrow$)	0.9894 (0.0021 $\uparrow$)	0.9824 (0.0785 $\uparrow$)	0.9999 (0.0004 $\uparrow$)	0.7664 (0.0419 $\uparrow$)	0.9973 (0.0831 $\uparrow$)	0.9998 (0.0013 $\uparrow$)
FPR	0.0019 (0.0035 $\downarrow$)	0.0015 (0.0223 $\downarrow$)	0.0034 (0.0157 $\downarrow$)	0.0184 (0.0939 $\downarrow$)	0 (0.0001 $\downarrow$)	0.0001 (0.0001 $\downarrow$)	0.0027 (0.0001 $\downarrow$)	0.0001 (0.0001 $\downarrow$)
D_{la} (ms)	0.0555 (0.0535 $\uparrow$)	0.0265 (0.0246 $\uparrow$)	0.0354 (0.0338 $\uparrow$)	0.0332 (0.031 $\uparrow$)	0.0234 (0.0217 $\uparrow$)	0.0339 (0.0319 $\uparrow$)	0.0229 (0.0213 $\uparrow$)	0.0193 (0.0165 $\uparrow$)

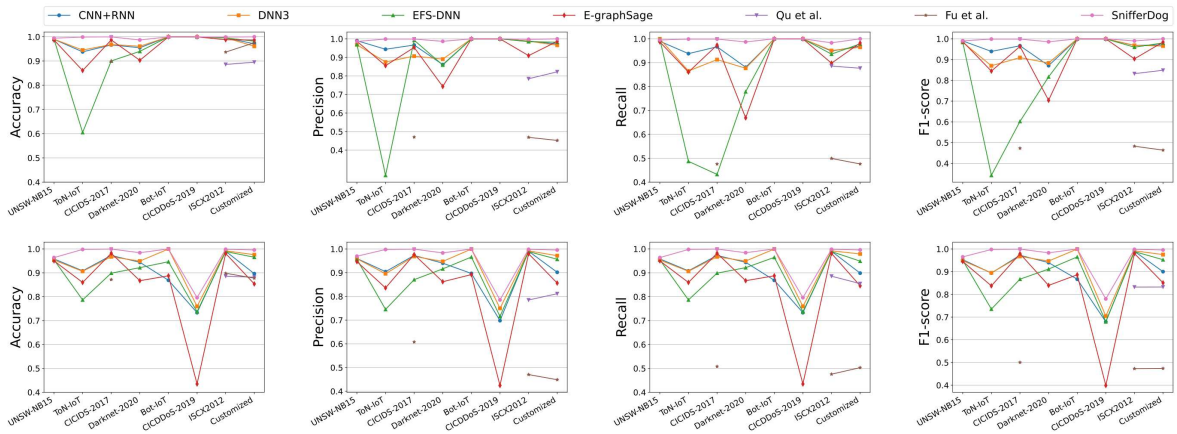


Fig. 3. Plots of accuracy, recall and F1 score metrics on the datasets. (Upper: binary classification; Bottom: multiclass classification).

false positive rate (FPR), Matthews Correlation Coefficient (MCC), and detection latency of SnifferDog are compared with the baselines. As illustrated in Table VII, SnifferDog achieves the best performance in terms of both MCC and FPR, with all multi-class FPRs being below 0.02 and seven of them even below 0.002. The best detection latency is achieved by EFS-DNN, which contains the fewest parameters and does not process the graph structure. SnifferDog's detection latency is close to the second best, DNN3.

Compared with the method proposed by Qu et al. [12], SnifferDog performs better across all metrics on the CICIDS-2017, ISCX-2012 and customized datasets. Both ISCX-2012 and customized datasets contain HTTP-based attacks, and SnifferDog still achieves better results for them. This comparison indicates that, for NIDS, the data-centric designation of deep learning model or architecture may be more effective than simply orchestrating or assembling multiple models.

The traditional machine learning-based method [24] shows undesirable results with low values (smaller than 50%) of precision, recall and F1 score on ISCX-2012 dataset. In the appendix of their paper, the F1 score of ISCX-2012 achieved

TABLE VIII
CROSS DOMAIN ANALYSIS

Dataset	Accuracy	Precision	Recall	F1-score
CICIDS-2017	0.9974	0.9974	0.9974	0.9974
ISCX-2012	0.9931	0.9931	0.9931	0.9921

95%; while in our experiment using their open source github code, the result is smaller than 50%. Anyway, even compared to their claimed value of 95%, our result of beyond 99% is better.

C. Feature Analysis

This experiment is carried out to measure the effects of different features. We analyze their effects by comparing the 77 statistical features extracted from the flows, and our flow encoding and topology features. CICIDS-2017, ISCX-2012 and customized datasets are used here.

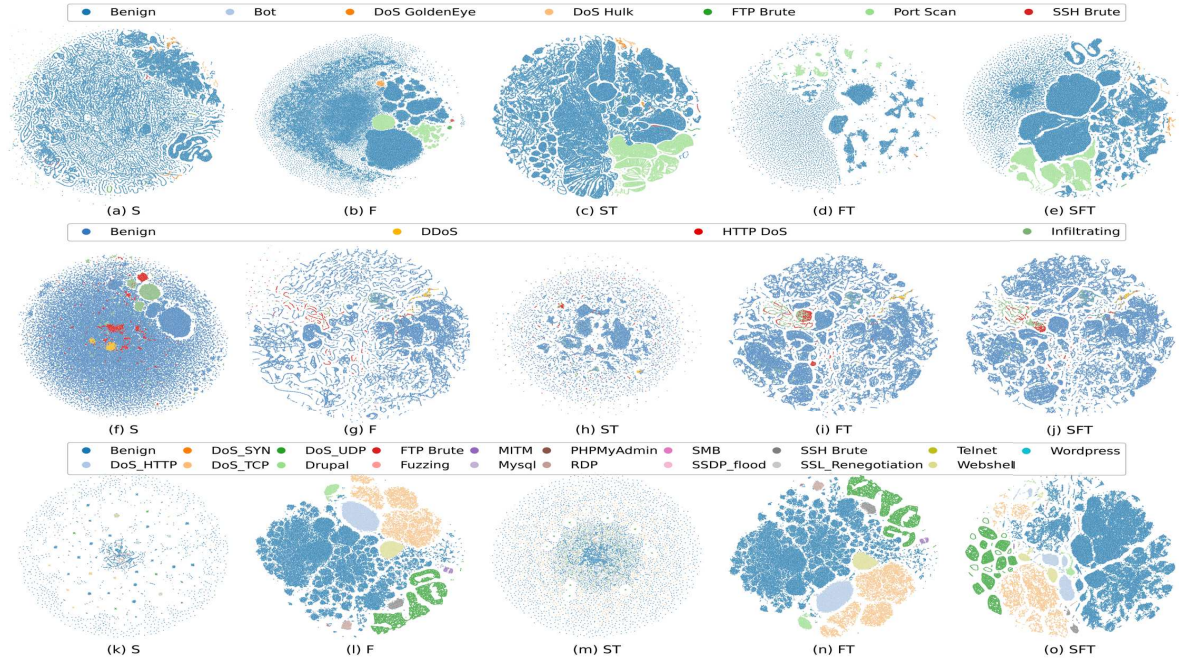


Fig. 4. Visualization of different features in multiclass task on CICIDS-2017 (upper), ISCX-2012 (middle) and customized (bottom) datasets, where S denotes statistical features, F denotes flow encoding and T denotes topology feature.

TABLE IX

EFFECT OF DIFFERENT FEATURES. THE BEST METRICS ARE HIGHLIGHTED IN BOLD

Feature	CICIDS-2017		ISCX-2012		Customized	
	A	F1	A	F1	A	F1
S	0.9995	0.9979	0.9956	0.9657	0.7730	0.7241
F	0.9997	0.9988	0.9976	0.9815	0.9916	0.9917
ST	0.9997	0.9989	0.9975	0.9806	0.7874	0.7355
FT	0.9997	0.9991	0.9992	0.9938	0.9998	0.9999
SFT	0.9997	0.9990	0.9989	0.9915	0.9994	0.9994

Table IX shows the results of these feature combinations, i.e., statistical only (S), flow encoding only (F), statistical and topology feature (ST), flow encoding and topology feature (FT), and all the three types (SFT), in order. The fusion of statistical features and topology feature is implemented by initializing the edges with statistical features, and all the classifiers are trained based on random forest algorithm. As we can see, the flow encoding and topology feature, i.e., the features used in our system, achieves all the best metrics. The statistical features indeed confound the classifiers, as there is a decrease in performance when they are combined with the flow encoding and topology features. This also demonstrates that encoding the raw flow data of network traffic and combining it with network topology for flow graph learning can achieve more effective detection without the need for artificially constructed statistical features. This approach reduces the process of manually extracting and designing features, as well as the errors associated with this process. In brief, the feature analysis experiments demonstrate that effective feature expression of network traffic can be achieved through raw traffic information without the need for artificially constructed features, with the network topology relationship enhancing the feature expression at the flow level.

In Figure 4, we utilize sample data and edge embeddings, followed by employing t-SNE (t-Distributed Stochastic Neighbor Embedding) and UMAP (Uniform Manifold Approximation and Projection) dimensionality reduction algorithms to map high-dimensional data into an easily understandable two-dimensional form for visual presentation. We visualize all the five feature sets for the three datasets. It is highly obvious that the plots of flow encoding and topology feature show more clear separations than others. The statistical plots undoubtedly show the worst result. The excellent flow encoding plots indicate that our flow encoding module can work effectively without topology learning. In summary, all these results show the effectiveness of our heterogeneous feature learning method and illustrate that our flow encoding result is a perfect substitute for statistical features.

We have also shown the wonderful effectiveness of cross-domain application of our flow pretraining component, in which we train the model using the customized benign traffic and apply it to other two datasets. As shown in Table VIII, all the metrics exceed 99%, which are even better than the result shown in Table IX. This result illustrates that the flow pretraining component significantly facilitates the real-world cross-domain applications. For example, it can be used to correlate compromised machines of a botnet in networks managed by different institutes, while preserving privacy by only correlating the flow embeddings.

D. Ablation Experiment

To analyze the effectiveness of packet-to-packet, flow-to-flow and flow-to-topology learning modules, we conducted multi-class experiments on the ToN-IoT, Bot-IoT, and CICDDoS-2019 datasets. These datasets were selected because they showed the worst results for both the baselines and SnifferDog, making them ideal for highlighting the differences in ablation analysis. We evaluated our topology embedding

TABLE X

RESULT OF ABLATION EXPERIMENT OF FLOW ENCODING. THE BEST METRICS ARE HIGHLIGHTED IN BOLD

Method	CICIDS-2017		ISCX-2012		Customized	
	A	F1	A	F1	A	F1
Attention	0.8827	0.8277	0.9356	0.9045	0.6092	0.5125
LSTM	0.9963	0.9962	0.9972	0.9965	0.9937	0.9937
Both	0.9974	0.9974	0.9974	0.9969	0.9929	0.9931

TABLE XI

RESULT OF ABLATION EXPERIMENT OF TOPOLOGY EMBEDDING. THE BEST METRICS ARE HIGHLIGHTED IN BOLD

Method	ToN-IoT		Darknet-2020		CICDDoS-2019	
	A	F1	A	F1	A	F1
n2n	0.9867	0.9868	0.9796	0.9786	0.7681	0.7387
n2e	0.9968	0.9968	0.9847	0.9838	0.7819	0.7632
n2e+n2n	0.9989	0.9989	0.9869	0.9858	0.7962	0.7803

model with and without attention layers, and utilized the three pcap datasets to measure the flow encoding model with and without LSTM and attention layers. The reason we do not directly utilize the pcap datasets to measure the topology embedding model is that the flow encoding feature achieves high performance across the four evaluation metrics, leading to negligible differences in the ablation analysis.

Table X presents the accuracy and F1 score from the ablation study of the flow encoding module. When a single module is employed, the performance of the Attention-only module is suboptimal. This suggests that the mechanism, when adept at focusing on individual features, struggles to capture relational information within the flow sequence. In contrast, the LSTM-only module maintains an accuracy rate above 99% across the three datasets, demonstrating its effectiveness in handling flow sequences and indicating that the sequential order of packets provides valuable information. However, this approach does not achieve the best possible results. The combination of the attention mechanism with LSTM yields the best outcomes on all three datasets, with accuracy rates exceeding 99.60%. On the customized dataset, the performance is enhanced by 0.23% compared to using LSTM alone. This demonstrates that the integration of Attention and LSTM not only strengthens the intrinsic features of the data but also effectively captures the interconnections between flow sequences.

In the flow graph learning part, the ablation study comparing n2n, n2e, and their combined approach is shown in Table XI. It is evident from the accuracy and F1 scores that n2n and n2e aggregation of information is crucial. The n2n method outperforms the n2e method, particularly on the complex CICDDoS-2019 dataset. While methods achieve the accuracy and F1 score above 73%, the n2e method shows a 1.76% improvement in accuracy and a 3.21% improvement in F1 score compared with the n2n method. This suggests that aggregating edge information provides more effective insights for detection. This is reasonable because most open-source datasets include single-step attacks, such as DoS and brute-force attacks, which involve rich flow-to-flow interactions with numerous correlated behaviors. Combining n2n and n2e methods enables a better capture of both inter-node interac-

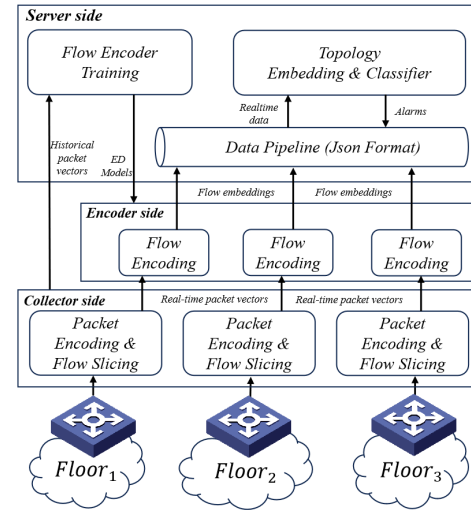


Fig. 5. Real-world deployment architecture.

tion information and associations between nodes and edges, thereby positively impacting detection performance.

E. Runtime Performance

We conducted a comparison test between our packet padding process and nPrint with 8 concurrent threads on a machine equipped with an Intel(R) Xeon(R) Silver 4314 CPU and a 16GB memory. SnifferDog pads 7.5 million packets in 69 seconds, achieving an average speed of 108,695 packets per second, about 12.45 times of nPrint (about 8,731 packets per second). This proves that SnifferDog is more suitable for real-time processing tasks.

We also compared the performance of our system with that of Qu et al. [12], whose work has a similar purpose—to implement nearly end-to-end feature representation of network traffic. The results are listed in Table XII. Their method extracts only flow features unless processing HTTP traffic, in which they aggregate the flows according to HTTP traces. Our flow pretraining process takes 46.8, 120.8 and 137.3 seconds for training on the three datasets, respectively, while 0.6, 2 and 2.2 seconds for encoding. In contrast, their work spends 67.5, 313.2 and 319 seconds for testing, respectively, with their training phase taking even longer. Overall, our approach is much more efficient than theirs in flow feature learning. In the topology feature learning and classification phases, i.e., the process of our flow graph learning component, SnifferDog spends 51.3, 597.3 and 259.4 seconds on the three datasets, respectively, classifying about 4,300 flows per second.

F. Real-World Deployment

To evaluate the effectiveness and efficacy in the real-world environment, we deployed SnifferDog in our institute's network starting on Nov. 1st, 2023. As briefly mentioned in Section V-A, this network consists of about 1,800 active IP addresses, serving devices like personal desktops, mobile phones, servers, and cameras. In a single day, there are about 20,000 outside IP addresses communicating with the internal ones, resulting in approximately 5,000 flows per hour and a total bandwidth usage of about 1.2 Gbps.

TABLE XII
PERFORMANCE COMPARISON

Model	Dataset	Extracting flow feature		Extracting topology feature		Total Parameters
		training(s)	testing(s)	training(s)	testing(s)	
Qu et al. [12]	ISCX-2012	51422.6	67.5	-	-	0.4M
	CIC-IDS-2017	>7d	313.2	-	-	
	Customized	>7d	319	-	-	
SnifferDog	ISCX-2012	46.8	0.6	76.1	51.3	0.26M
	CIC-IDS-2017	120.8	2	438.5	597.3	
	Customized	137.3	2.2	183.9	259.4	

In Figure 5 presents the deployment architecture for this experiment, which includes the collector, encoder and server sides. These components generate uniform packet vectors and flow embeddings, train models, and classify malicious flows, respectively. The office area of our institute covers three floors ($floor_1$, $floor_2$ and $floor_3$), within a building. Several switches have been installed in the low-voltage room on each floor to manage the network communication. These rooms are too small to place servers and lack cooling equipment. Therefore, we deployed three desktops with an Intel(R) Xeon(R) Silver 4314 CPU and a 64GB memory on each floor. These desktops are used to carry out both the packet and flow encoding components and transfer results to the GPU server center. The GPU server center contains three machines, each equipped with an Intel Xeon Gold5318Y@2.10GHz 24Cores x2 CPU, an NVIDIA A800 GPU card and a 256GB memory. They perform flow encoder training as well as the topology embedding and classification tasks. The historical packet vectors are collected directly from client side to server side, while the real-time data is delivered in json format through a distributed data pipeline (i.e., Kafka-3.5), which is served by five machines with an Intel Xeon Gold5318Y@2.10GHz 24Cores x2 CPU, and a 256GB memory. Alarms are send back to the pipeline for visualization in the UI modules, which are not detailed here.

The flow encoding model was trained based on data collected on Oct. 30th and 31st, 2023 from $floor_2$, with about 0.7 million (benign) flows. This means during this period, no attacks were detected in our institute's network, likely due to the deployment of multiple IDS and EDR (Endpoint Detection and Response) modules at the network border and on critical machines. Additionally, the users—students and faculty—possess a strong security awareness, making unauthorized access to the network difficult. The clean environment provides an ideal setting for evaluating the false positive rate (FPR) of SnifferDog, in which any alarms are false positives. The FPR represents the ratio of misclassified normal flows, which is a crucial metric for assessing real-world usability. At the beginning, the FPR of our system is about 0.08%, yet it rises to approximately 3% after a month in Nov. 30th, 2023. This introduces a data drift problem, where the data distribution changes as time goes by. Upon further investigation, this shift was found to be caused by the afterwards installed applications in our institute. For example, the frequent communication generated by the load balancing function of our kafka cluster led to a large portion of the false alarms, as the kafka cluster was not present during the training period. This issue is addressed by periodically retraining our models,

though this solution is not perfect. The concept drift has not been considered in this study and will be an interesting task for future work.

VI. CONCLUSION

This paper introduces SnifferDog, a novel flow detection system that comprehensively learns and fuses features involved in packets, flows and topology. It implements an efficient flow formatting process to ensure high throughput and extracts both the sequential and non-sequential relations of packets in a flow. Furthermore, a GNN model, adaptive to the flow graph format, is enhanced by n2n and n2e attention layers to integrate heterogeneous topological relations into flow vectors. The rich feature representation ability enables accurate identification of malicious behaviors. Experimental results demonstrate SnifferDog's excellent effectiveness across different environments and its practical usability in real-world scenarios.

REFERENCES

- [1] Z. Xu, P. Fang, C. Liu, X. Xiao, Y. Wen, and D. Meng, "DEPCOMM: Graph summarization on system audit logs for attack investigation," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Francisco, CA, USA, May 2022, pp. 540–557.
- [2] M. H. Nasir, S. A. Khan, M. M. Khan, and M. Fatima, "Swarm intelligence inspired intrusion detection systems—A systematic literature review," *Comput. Netw.*, vol. 205, Mar. 2022, Art. no. 108708.
- [3] L. Vu, Q. U. Nguyen, D. N. Nguyen, D. T. Hoang, and E. Dutkiewicz, "Deep generative learning models for cloud intrusion detection systems," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 565–577, Jan. 2023, doi: [10.1109/TCYB.2022.3163811](https://doi.org/10.1109/TCYB.2022.3163811).
- [4] D. Chen, F. Zhang, and X. Zhang, "Heterogeneous IoT intrusion detection based on fusion word embedding deep transfer learning," *IEEE Trans. Ind. Informat.*, vol. 19, no. 8, pp. 9183–9193, Aug. 2023, doi: [10.1109/TII.2022.3227640](https://doi.org/10.1109/TII.2022.3227640).
- [5] I. Mohammed Sayem, M. Islam Sayed, S. Saha, and A. Haque, "ENIDS: A deep learning-based ensemble framework for network intrusion detection systems," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 5, pp. 5809–5825, Oct. 2024, doi: [10.1109/TNSM.2024.3414305](https://doi.org/10.1109/TNSM.2024.3414305).
- [6] Z. Yang et al., "A systematic literature review of methods and datasets for anomaly-based network intrusion detection," *Comput. Secur.*, vol. 116, May 2022, Art. no. 102675.
- [7] M. Nakip and E. Gelenbe, "Online self-supervised deep learning for intrusion detection systems," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5668–5683, 2024, doi: [10.1109/TIFS.2024.3402148](https://doi.org/10.1109/TIFS.2024.3402148).
- [8] M. Ge, N. F. Syed, X. Fu, Z. Baig, and A. Robles-Kelly, "Towards a deep learning-driven intrusion detection approach for Internet of Things," *Comput. Netw.*, vol. 186, Feb. 2021, Art. no. 107784.
- [9] A. Chen, Y. Fu, X. Zheng, and G. Lu, "An efficient network behavior anomaly detection using a hybrid DBN-LSTM network," *Comput. Secur.*, vol. 114, Mar. 2022, Art. no. 102600.
- [10] E. Min, J. Long, Q. Liu, J. Cui, and W. Chen, "TR-IDS: Anomaly-based intrusion detection through text-convolutional neural network and random forest," *Secur. Commun. Netw.*, vol. 2018, pp. 1–9, Jul. 2018.

- [11] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, Budapest, Hungary, Apr. 2022, pp. 1–9.
- [12] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, J. A. Calandrino and C. Troncoso, Eds., 2023, pp. 589–606.
- [13] B. M. Aslahi-Shahri et al., "A hybrid method consisting of GA and SVM for intrusion detection system," *Neural Comput. Appl.*, vol. 27, no. 6, pp. 1669–1676, Aug. 2016.
- [14] I. Sumaiya Thaseen and C. Aswani Kumar, "Intrusion detection model using fusion of chi-square feature selection and multi class SVM," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 29, no. 4, pp. 462–472, Oct. 2017.
- [15] M. A. Ambusaidi, X. He, P. Nanda, and Z. Tan, "Building an intrusion detection system using a filter-based feature selection algorithm," *IEEE Trans. Comput.*, vol. 65, no. 10, pp. 2986–2998, Oct. 2016.
- [16] H. Alazzam, A. Sharieh, and K. E. Sabri, "A feature selection algorithm for intrusion detection system based on pigeon inspired optimizer," *Expert Syst. Appl.*, vol. 148, Jun. 2020, Art. no. 113249.
- [17] A. H. Engly, A. R. Larsen, and W. Meng, "Evaluation of anomaly-based intrusion detection with combined imbalance correction and feature selection," in *Proc. Int. Conf. Netw. Syst. Secur.*, 2020, pp. 277–291.
- [18] X. Li, W. Chen, Q. Zhang, and L. Wu, "Building auto-encoder intrusion detection system based on random forest feature selection," *Comput. Secur.*, vol. 95, Aug. 2020, Art. no. 101851.
- [19] Y. Zhou, G. Cheng, S. Jiang, and M. Dai, "Building an efficient intrusion detection system based on feature selection and ensemble classifier," *Comput. Netw.*, vol. 174, Jun. 2020, Art. no. 107247.
- [20] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. V. Ramos, and A. Madeira, "FlowLens: Enabling efficient flow classification for ML-based network security applications," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021.
- [21] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. WWW, F. Laforest, R. Troncy, E. Simperl, D. Agarwal, A. Gionis, I. Herman, and L. Médini, Eds.*, Apr. 2022, pp. 633–642, doi: [10.1145/3485447.3512217](https://doi.org/10.1145/3485447.3512217).
- [22] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Austin, TX, USA, Apr. 2023, pp. 2066–2075, doi: [10.1145/3543507.3583227](https://doi.org/10.1145/3543507.3583227).
- [23] T. Cui et al., "TrafficLLM: Enhancing large language models for network traffic analysis with generic traffic representation," 2025, *arXiv:2504.04222*.
- [24] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, Feb. 2023.
- [25] N. Bonelli, S. Giordano, and G. Prociassi, "Enabling packet fan-out in the libpcap library for parallel traffic processing," in *Proc. Netw. Traffic Meas. Anal. Conf. (TMA)*, Dublin, Ireland, Jun. 2017, pp. 1–9.
- [26] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.
- [27] Y. LeCun, Y. Bengio, and G. E. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015, doi: [10.1038/nature14539](https://doi.org/10.1038/nature14539).
- [28] Xterminal. Accessed: Jul. 10, 2024. [Online]. Available: <https://www.xterminal.cn/>
- [29] R. Mitchell and I.-R. Chen, "Behavior-rule based intrusion detection systems for safety critical smart grid applications," *IEEE Trans. Smart Grid*, vol. 4, no. 3, pp. 1254–1263, Sep. 2013.
- [30] S. Mabu, C. Chen, N. Lu, K. Shimada, and K. Hirasawa, "An intrusion-detection model based on fuzzy class-association-rule mining using genetic network programming," *IEEE Trans. Syst., Man, Cybern., C Appl. Rev.*, vol. 41, no. 1, pp. 130–139, Jan. 2011.
- [31] R. M. A. Mohammad, M. K. Alsmadi, I. Almarashdeh, and M. Alzaqebah, "An improved rule induction based denial of service attacks classification model," *Comput. Secur.*, vol. 99, Dec. 2020, Art. no. 102008.
- [32] M. Moghimi and A. Y. Varjani, "New rule-based phishing detection method," *Expert Syst. Appl.*, vol. 53, pp. 231–242, Jul. 2016.
- [33] Q. Yang, J. Yang, W. Yu, D. An, N. Zhang, and W. Zhao, "On false data-injection attacks against power system state estimation: Modeling and countermeasures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 717–729, Mar. 2014.
- [34] J. Sinha and M. Manollas, "Efficient deep CNN-BiLSTM model for network intrusion detection," in *Proc. 3rd Int. Conf. Artif. Intell. Pattern Recognit.*, Jun. 2020, pp. 223–231.
- [35] G. Andresini, A. Appice, and D. Malerba, "Nearest cluster-based intrusion detection through convolutional neural networks," *Knowl.-Based Syst.*, vol. 216, Mar. 2021, Art. no. 106798.
- [36] R. K. Vigneswaran, R. Vinayakumar, K. Soman, and P. Poornachandran, "Evaluating shallow and deep neural networks for network intrusion detection systems in cyber security," in *Proc. 9th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Oct. 2018, pp. 1–6.
- [37] S. Baldoni and F. Battisti, "Histogram-based network traffic representation for anomaly detection through PCA," *Comput. Netw.*, vol. 265, Jun. 2025, Art. no. 111276, doi: [10.1016/j.comnet.2025.111276](https://doi.org/10.1016/j.comnet.2025.111276).
- [38] E. Akpaku, J. Chen, M. Ahmed, F. K. Agbenyegah, and W. L. Brown-Acuaye, "RAGN: Detecting unknown malicious network traffic using a robust adaptive graph neural network," *Comput. Netw.*, vol. 262, May 2025, Art. no. 111184, doi: [10.1016/j.comnet.2025.111184](https://doi.org/10.1016/j.comnet.2025.111184).
- [39] Y. Wang, Y. Jiang, and J. Lan, "FCNN: An efficient intrusion detection method based on raw network traffic," *Secur. Commun. Netw.*, vol. 2021, pp. 1–13, Jun. 2021.
- [40] B. Wang, Y. Su, M. Zhang, and J. Nie, "A deep hierarchical network for packet-level malicious traffic detection," *IEEE Access*, vol. 8, pp. 201728–201740, 2020.
- [41] Z. Wang, J. Liu, and L. Sun, "EFS-DNN: An ensemble feature selection-based deep learning approach to network intrusion detection system," *Secur. Commun. Netw.*, vol. 2022, pp. 1–14, Apr. 2022.
- [42] J. Xie, S. Li, X. Yun, Y. Zhang, and P. Chang, "HSTF-model: An HTTP-based trojan detection model via the hierarchical spatio-temporal features of traffics," *Comput. Secur.*, vol. 96, Sep. 2020, Art. no. 101923.
- [43] C. Fu, Q. Li, M. Shen, and K. Xu, "Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 3659–3673, doi: [10.1145/3658644.3670353](https://doi.org/10.1145/3658644.3670353).
- [44] W. Wang et al., "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.
- [45] N. Nethercote and J. Seward, "Valgrind: A program supervision framework," in *Proc. 3rd Workshop Run-Time Verification, RV@CAV*, Jul. 2003, pp. 44–66.
- [46] G. Combette and G. Munch-Maccagnoni, "A resource modality for RAIL," in *Proc. LOLA: Workshop Syntax Semantics Low-Level Lang.*, 2018, pp. 1–4.
- [47] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 1024–1034.
- [48] J. Khoury, D. Klisura, H. Zanddizari, G. De La Torre Parra, P. Najafirad, and E. Bou-Harb, "Jbeil: Temporal graph-based inductive learning to infer lateral movement in evolving enterprise networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 3644–3660, doi: [10.1109/SP54263.2024.00009](https://doi.org/10.1109/SP54263.2024.00009).
- [49] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MILCIS)*, Nov. 2015, pp. 1–6.
- [50] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 102994.
- [51] A. Habibi Lashkari, G. Kaur, and A. Rahali, "DIDarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning," in *Proc. 10th Int. Conf. Commun. Netw. Secur.*, Nov. 2020, pp. 1–13.
- [52] J. Ashraf et al., "IoTBoT-IDS: A novel statistical learning-enabled botnet detection framework for protecting networks of smart cities," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 103041.
- [53] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICST)*, Oct. 2019, pp. 1–8.
- [54] A. Yulianto, P. Sukarno, and N. A. Suwastika, "Improving AdaBoost-based intrusion detection system (IDS) performance on CIC IDS 2017 dataset," *J. Phys., Conf. Ser.*, vol. 1192, Mar. 2019, Art. no. 012018.
- [55] A. Shiravi, H. Shiravi, M. Tavallae, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Comput. Secur.*, vol. 31, no. 3, pp. 357–374, May 2012.
- [56] CICFlowMeter. Accessed: Oct. 27, 2025. [Online]. Available: <https://github.com/ahlashkari/CICFlowMeter>

- [57] *Ixia Perfectstorm*. Accessed: Jul. 12, 2024. [Online]. Available: <https://www.keysight.com/us/en/products/network-test/network-test-hardware/perfectstorm.html>
- [58] *Argus*. Accessed: Jul. 12, 2024. [Online]. Available: <https://openargus.org/>
- [59] B. Chen, J. Lee, and A. S. Wu, "Active event correlation in bro IDS to detect multi-stage attacks," in *Proc. 4th IEEE Int. Workshop Inf. Assurance (IWIA)*, Apr. 2006, pp. 32–50.
- [60] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, Portugal, Jan. 2018, pp. 108–116.
- [61] V. Labayen, E. Magaña, D. Morató, and M. Izal, "Online classification of user activities using machine learning on network traffic," *Comput. Netw.*, vol. 181, Nov. 2020, Art. no. 107557.
- [62] G. Oluchi Anyanwu, C. I. Nwakanma, J.-M. Lee, and D.-S. Kim, "Optimization of RBF-SVM kernel using grid search algorithm for DDoS attack detection in SDN-based VANET," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8477–8490, May 2023.



Xi Luo received the Ph.D. degree in computer science and technology from the Institute of Information Engineering, Chinese Academy of Sciences, in 2019. He is currently an Associate Professor of network security with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. He has authored publications published by refereed international journals and conferences, e.g., ICDE, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE TRANSACTIONS ON RELIABILITY, and the High Performance Computing and Communications. His research interests include AI-enabled threat analysis, the Internet of Things security, and network anomaly detection.



Lihua Yin received the Ph.D. degree from Harbin Institute of Technology, Harbin, China. She is currently a Professor and the Associate Dean of the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. She has authored more than 140 papers published by refereed international conferences and journals, e.g., IEEE TRANSACTIONS ON KNOWLEDGE DATA ENGINEERING, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, *Computers and Security*, and *Infocom*. She has been the Project Leader of the National Natural Science Foundation of China, the National Key Research and Development Plan of China, the Major State Basic Research Development Program of China, and the National High-Tech Research and Development Program of China. Her research interests include network security, the Internet of Things, secure threat intelligence sharing, and big data. She is an Editorial Board Member of *Journal of Network and Information Security*.



Hongyu Yang received the B.S. degree in Internet of Things engineering from the North University of China, Taiyuan, China, in 2022, and the M.S. degree in network and information security from the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China, in 2025, where he is currently pursuing the Ph.D. degree in cyberspace security. His research interests include network intrusion detection and graph neural networks.



Zeyan Liu received the bachelor's degree in math from Wuhan University in 2019 and the Ph.D. degree in computer science from The University of Kansas in 2024 advised by Dr. Bo Luo and Dr. Fengjun Li. He is currently a tenure-track Assistant Professor at CSE, University of Louisville. His research interests include cybersecurity and AI, including adversarial and privacy-preserving machine learning, AI safety, accountability, fairness, explainability, and transparency (SAFE-T), and AI for cybersecurity.



Weizhe Chen is currently pursuing the Ph.D. degree with the Cyberspace Institute of Advanced Technology, Guangzhou University, China. His research focuses on applying deep learning techniques to address challenges in cyberspace security, with specific interests in data security, federated learning, and network intrusion detection.



Shijie Jia received the Ph.D. degree in information security from the University of Chinese Academy of Sciences in 2017. He is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include information system security and cryptographic application security.



Bo Luo (Member, IEEE) received the Ph.D. degree from The Pennsylvania State University in 2008. He is currently a Professor with the Institute for Information Sciences and the Department of Electrical Engineering and Computer Science, The University of Kansas. His current research interests include the intersection of security and privacy and AI/ML.



Hongli Xiang received the M.S. degree from the Cyberspace Institute of Advanced Technology, Guangzhou University, China. His expertise lies in network-traffic analytics and deep learning, with a focus on building robust intrusion detection systems (IDS). He now applies these skills to real-world cybersecurity challenges, identifying and mitigating network intrusions.